

Solar System Technical Report

Generated for the Solar System Unity project.

1. Project Overview

This project is a Unity 6 Universal Render Pipeline application that presents an explorable solar system with a custom orbital simulation, interactive camera controls, scientific metadata panels, a tactical minimap, asteroid belt rendering, and stylized Lagrange-point missions.

The implementation is physically inspired rather than strictly astrophysical. Relative sizes and distances are grounded in real-world values, while gravity, time, and some motion systems are normalized so the experience remains stable and usable inside Unity.

2. Technology Stack

- Engine: Unity 6 (6000.4.0f1)
- Rendering: Universal Render Pipeline
- UI: uGUI and TextMeshPro
- Input: Unity Input System enabled, with legacy input support still used by several runtime scripts
- Language: C#
- Runtime structure: custom simulation layer instead of Unity Rigidbody orbital physics

3. Core Architecture

The project is organized around a small set of core runtime responsibilities:

- `SimulationManager`: advances all orbital state
- `GravityBody`: stores simulation state for a single simulated object
- `CelestialBody`: stores descriptive metadata and hierarchy links used by the UI
- `CameraFollow`: handles free flight, autopilot approach, and orbital viewing
- `SelectionManager`: coordinates object selection, camera targeting, and UI state
- `ScientificDetailsUI`: renders the animated scientific panel
- `MinimapController`: renders the 2D tactical map
- `SolarSystemGenerator`: editor-side content generation for the main system

The most important architectural decision is the separation between simulation state and scene presentation. Positions and velocities are computed in custom data objects and then written back to Unity transforms after each simulation step.

4. Spatial Scale Model

The project uses a fixed spatial normalization:

1 Unity unit = 100,000 km

This is implemented in `SolarSystemScale.cs`.

The conversion functions are:

```
units = kilometers / 100000  
kilometers = units * 100000
```

Examples:

- Earth diameter: 12,742 km -> 0.12742 units
- Earth average distance from Sun: 149,600,000 km -> 1496 units
- Sun diameter: 1,392,700 km -> 13.927 units

This means the project preserves relative spatial proportions while compressing astronomical values into a range Unity can render and manipulate reliably.

5. Why Raw Real-World Values Are Not Used

Using literal astronomical scales and literal SI-unit physics inside Unity is impractical for several reasons:

- Unity transform positions are float-based, so extremely large distances lose precision
- Rendering and interaction become unstable when bodies are very far from the origin
- Real-world orbital scales make planets visually tiny compared with their distances
- Real SI-unit gravitational values are not convenient for frame-based interactive simulation

Because of that, the project uses normalized space, normalized time, and a tuned gravity constant while preserving believable spatial ratios.

6. Simulation Model

The orbital system is implemented in `SimulationManager.cs`.

6.1 Simulation State

Each `GravityBody` contains:

- double-precision position
- double-precision velocity
- double-precision acceleration
- scalar mass

Double precision is provided through `Vector3d`, which reduces large-scale drift and precision loss compared with keeping all orbital math in Unity `Vector3`.

6.2 Time Advancement

Each physics frame computes:

```
totalElapsed = Time.fixedDeltaTime * simulationTimeStep  
dt = totalElapsed / subSteps
```

The simulation then performs multiple smaller substeps per frame. This improves stability and reduces integration error.

6.3 Integrator

The runtime uses a velocity-Verlet style update:

```

position = position + velocity * dt + 0.5 * acceleration_old * dt^2
velocity = velocity + 0.5 * acceleration_old * dt
recompute acceleration
velocity = velocity + 0.5 * acceleration_new * dt

```

This is a strong choice for orbital motion because it is more stable than a naive Euler update while remaining simple to implement.

6.4 Gravity Computation

For every body A, acceleration is accumulated from all other bodies B:

```

direction = position_B - position_A
distSq = |direction|^2
acceleration += normalize(direction) * (G * mass_B / distSq)

```

Where:

- G is the tuned gravitational constant used by the project
- mass_B is the normalized mass of the other body
- distSq is the squared separation in simulation units

The implementation also protects against singularities by skipping extremely small distances.

7. Initial Orbit Construction

The initial solar system layout is generated in `SolarSystemGenerator.cs`.

Planet starting velocities are initialized using a circular-orbit approximation:

```
v = sqrt(G * M / r)
```

Where:

- G is the project gravity constant
- M is the mass of the central body
- r is orbital radius in simulation units

Moon initial velocities are computed similarly, but relative to their parent planet and then added to the parent body's velocity.

8. Barycenter Correction

After all bodies are created, the generator computes the total momentum of all non-solar bodies:

```

totalMomentum = sum(velocity_i * mass_i)
sunVelocity = -totalMomentum / sunMass

```

This gives the sun an offsetting initial velocity so the system better respects the overall barycenter instead of treating the sun as an immovable origin.

That is an important detail because it prevents the rest of the system from carrying net momentum with no balancing response.

9. Rotation Model

Visual self-rotation is applied separately from orbital translation.

For each body:

```
rotationSpeed = 360 / rotationPeriodDays
```

The body is then rotated around its local up axis every update after orbital position is applied.

This is a presentation-oriented layer rather than part of the orbital solver.

10. Time Control Strategy

`TimeController.cs` changes `Time.timeScale`, but intentionally keeps `Time.fixedDeltaTime` at its original value.

This means that as time scale increases, Unity processes more fixed-step simulation work per real second instead of increasing the duration of each physics step. That preserves orbit stability much better than scaling both values together.

This is a strong practical compromise for interactive orbital visualization.

11. Camera System

`CameraFollow.cs` supports three modes:

- manual free flight
- autopilot approach
- locked orbital viewing

11.1 Manual Flight

The player uses standard local-space movement and mouse look:

- WASD for planar movement
- Q and E for vertical movement
- right mouse drag for view rotation
- Left Shift for speed boost

11.2 Autopilot

Autopilot works by preserving a stable approach direction and smoothing the camera's current distance toward a target stop distance:

```
stopDistance = targetScale * arrivalDistanceMultiplier
currentDistance = SmoothDamp(currentDistance, stopDistance)
cameraPosition = targetPosition + approachDirection * currentDistance
cameraRotation = Slerp(currentRotation, LookRotation(target - camera))
```

This avoids a rotation-position feedback loop and produces a much cleaner arrival sequence than constantly recomputing a moving orbit path during approach.

11.3 Locked Orbit

Once the camera arrives, the system switches into an orbital inspection mode driven by yaw, pitch, and distance:

```
rotation = Euler(pitch, yaw, 0)
offset = rotation * (0, 0, -distance)
cameraPosition = targetPosition + offset
```

This gives the user a stable “study mode” around a selected object.

12. Selection and UI Flow

The selection pipeline is:

- player raycasts into the scene
- hit object resolves to a `CelestialBody`
- `SelectionManager` sends the camera to the target
- `ScientificDetailsUI` prepares hidden content during flight
- on autopilot arrival, the UI panel animates in and types out the metadata

This keeps navigation and information display synchronized, which improves perceived polish.

13. Scientific Metadata Model

`CelestialBody.cs` and the JSON data assets in `Assets/Celestial Bodies Data` provide the descriptive layer for the project.

The metadata includes:

- mass
- diameter
- orbital period
- temperature
- velocity
- density
- axial tilt
- eccentricity
- atmospheric and compositional notes

The generated scene uses this metadata for the scientific sidebar, while the simulation itself uses normalized values from the editor generator.

14. Minimap System

The minimap is implemented in `MinimapController.cs`.

Main features:

- zoomable 2D solar system map
- focus-lock to selected target
- orbit rings
- player/camera marker
- marker fading for small bodies
- scale bar
- moon and satellite toggles

14.1 Zoom

Zoom is smoothed exponentially:

```
currentZoom = exp( lerp(log(currentZoom), log(targetZoom), smoothing) )
```

This gives a more consistent feel across huge scale ranges than linear interpolation.

14.2 Marker Placement

Bodies are projected into minimap space using their x and z world coordinates:

```
markerPosition = (body.x, body.z) * currentZoom
```

14.3 Scale Bar

The minimap scale bar estimates physical distance using the same spatial normalization:

```
kmPerPixel = 100000 / currentZoom  
totalKm = kmPerPixel * targetPixelWidth
```

The displayed text is then formatted as kilometers, millions of kilometers, or astronomical units depending on magnitude.

15. Asteroid Belt Rendering

The asteroid belt is implemented in `AsteroidBeltGenerator.cs`.

Instead of creating thousands of ordinary moving `GameObjects`, the system:

- generates per-asteroid orbit parameters
- packs them into batched arrays
- uses `Graphics.DrawMeshInstanced`
- drives orbital motion in the shader using per-instance parameters

The angular speed approximation is:

```
orbitSpeed = sqrt(G * sunMass / radius) / radius
```

This is effectively an angular velocity term derived from a circular-orbit speed model.

The recent refactor pre-batches the instance data so it no longer reallocates orbit arrays every frame, which is a meaningful runtime improvement.

16. Lagrange Mission Approximation

The project includes stylized L1 and L2 missions around the Sun-Earth system in `LagrangeSatellite.cs`.

The workflow is:

- compute Sun-Earth direction
- place the L-point center about 1,500,000 km from Earth
- build a Lissajous-style local motion around that center

The local offsets are:

```
offsetX = sin(freq) * amplitude  
offsetZ = cos(freq) * amplitude * 0.5  
offsetY = sin(freq * 0.5) * amplitude * 0.3
```

These offsets are then oriented relative to the actual Sun-Earth axis. This is not a mission-grade dynamical model, but it gives a convincing stylized representation of halo-orbit behavior.

17. Visual Presentation Layers

Several systems are intentionally exaggerated for readability and atmosphere:

- orbit trails

- pulsing selection ring
- atmosphere shell rendering
- sun emission adjustment based on viewer distance
- minimap icons and orbit overlays

These are presentation systems rather than scientific simulation systems, and that separation is one of the strengths of the project.

18. Realism Assessment

The project is best described as:

`spatially realistic in ratio, dynamically stylized for usability`

That means:

- body size and distance relationships are based on real values
- time is compressed
- gravity is normalized
- orbital planes and mission motion are simplified
- UI and visibility systems are intentionally exaggerated

This is a correct and defensible engineering decision for Unity. Using literal astronomical values would create major precision, usability, and rendering problems.

19. Recent Maintainability Improvements

The latest refactor work improved the project without changing user-facing logic:

- centralized metadata loading
- centralized celestial-body lookup registry
- removed reflection-based HUD access
- reduced repeated global searches
- reduced asteroid-belt allocation churn
- added validation tooling and edit-mode tests
- added documentation for scaling and architecture

These changes make the project easier to maintain and easier to present in a portfolio or interview setting.

20. Recommended Talking Points

If this project is being shown in a portfolio, the strongest technical talking points are:

- custom orbital simulation using double precision
- normalized but physically grounded scale model
- velocity-Verlet integration
- barycenter momentum correction
- editor tooling for procedural setup

- instanced asteroid rendering
- synchronized camera-selection-UI flow

21. Conclusion

This is a strong technical portfolio project because it balances simulation, visualization, tooling, and usability. It does not attempt to force literal astrophysics into a game engine. Instead, it preserves the most important scientific relationships while adapting the system to Unity's numerical and interaction constraints.

That is not a shortcut. It is the correct design choice for this kind of interactive software.